

**BTS Services Informatiques aux Organisations (SIO)**  
***Session 2025***

**Nom du candidat :** Da Silva Peixoto

**Prénom du candidat :** Gabriel

**Parcours :** BTS SIO option SLAM

***Fiche de réalisation professionnelle N° 11***

**Structuration d'un environnement de développement  
professionnel**

**Période de réalisation :** 2025-2026

**Type de réalisation :** Durant la formation et durant mes stages.

**Mode de réalisation :** Individuelle

## **Sommaire**

|                                           |          |
|-------------------------------------------|----------|
| <b>Contexte organisationnel</b>           | <b>3</b> |
| <b>Problématique</b>                      | <b>3</b> |
| <b>Choix de la solution mise en œuvre</b> | <b>3</b> |
| <b>Résolution</b>                         | <b>3</b> |
| <b>Conclusions</b>                        | <b>4</b> |
| <b>Compétences</b>                        | <b>4</b> |

## CONTEXTE ORGANISATIONNEL

Au cours de mes deux expériences professionnelles, j'ai dû m'adapter à des écosystèmes technologiques opposés. Au Laboratoire TIMC (CNRS), j'ai participé à la création de Cardiac-TS, un client lourd exigeant la compilation de code orienté objet complexe en Java (JavaFX) et l'orchestration de conteneurs Docker. À l'inverse, chez Automatique & Industrie, j'ai intégré une équipe travaillant sur Holonet, ce qui nécessitait l'utilisation d'une stack Web moderne et asynchrone axée sur JavaScript, avec Node.js et le framework Vue.js (Nuxt). Malgré ces différences fondamentales de langages et d'architectures, j'ai dû relever un défi commun : être opérationnel, rigoureux et productif le plus rapidement possible au sein de ces deux environnements.

## PROBLÉMATIQUE

Un développeur qui utilise des outils inadaptés perd un temps précieux en configuration, au détriment de l'apprentissage et de la production logicielle. Tenter de forcer le développement d'un gros projet Java dans un éditeur de texte léger, ou à l'inverse, faire du développement Web front-end dans un IDE lourd conçu pour la machine virtuelle Java (JVM), génère d'énormes frictions (lenteurs, autocomplétion défaillante, mauvaise intégration des tests). L'enjeu de ma montée en compétences n'était donc pas de trouver un outil unique "à tout faire", mais de concevoir un espace de travail hybride. Il me fallait mettre en place un environnement de développement et d'apprentissage capable de m'offrir une assistance stricte pour le typage Java d'un côté, et une grande flexibilité pour Vue.js (chez A&I) de l'autre.

## CHOIX DE LA SOLUTION MISE EN ŒUVRE

Pour centraliser mes outils de développement de manière optimale, j'ai fait le choix stratégique de la spécialisation logicielle. Pour le développement back-end et logiciel au TIMC, j'ai opté pour **IntelliJ IDEA** (JetBrains). Cet Environnement de Développement Intégré (IDE) est la référence absolue pour l'écosystème Java, offrant une compréhension profonde du code source, une intégration native de Maven et des outils d'analyse de base de données puissants. En revanche, pour le développement Web chez Automatique & Industrie, j'ai fait le choix exclusif de **Visual Studio Code (VSCode)**. Ce choix s'explique par sa légèreté, son hégémonie dans le monde JavaScript/TypeScript, et son écosystème d'extensions communautaires indispensable pour s'aligner sur les standards de formatage d'une équipe Web moderne.

## RÉSOLUTION

La structuration de mon IDE s'est articulée autour de trois piliers fondamentaux pour mon travail quotidien chez Automatique & Industrie :

1. **Automatisation de la qualité et des standards (Linting & Formatage) :** J'ai couplé les extensions **ESLint** (pour l'analyse des règles logiques du JavaScript) et **Prettier** (pour le formatage visuel). J'ai configuré VSCode pour que le code soit formaté et corrigé automatiquement à chaque sauvegarde (`"editor.formatOnSave": true`). Cette configuration a été cruciale : elle garantissait que tout le code que je poussais sur GitLab respectait à 100 % les conventions de l'équipe Holonet, éliminant totalement les remarques liées au style lors des Pull Requests.
2. **Versioning et contexte collaboratif (GitLens) :** J'ai installé l'extension **GitLens**, qui superpose les données de Git directement dans l'interface de VSCode. À chaque ligne de code que je lisais dans l'application Holonet, je pouvais voir en filigrane (Git Blame) quel développeur de l'équipe avait écrit cette ligne et à quelle date. Cela m'a permis de cibler immédiatement la bonne personne lorsque j'avais besoin d'explications sur une fonctionnalité spécifique, fluidifiant ainsi la communication au sein de l'équipe.
3. **Intégration de l'Intelligence Artificielle (Le Copilote) :** L'intégration de l'IA générative dans VSCode a été un véritable levier de productivité. Lors de la migration technologique, je sélectionnais des blocs de l'ancien code en Go et je demandais à l'IA (Gemini) de m'expliquer la logique métier sous-jacente. Grâce à cette extension j'ai pu comprendre la logique du code plus rapidement.

## CONCLUSIONS

La construction de cet environnement VSCode sur-mesure a grandement facilité mon intégration chez Automatique & Industrie. En déléguant les tâches répétitives (formatage, détection d'erreurs de frappe) aux extensions, et la génération de code rébarbatif à l'Intelligence Artificielle, j'ai pu concentrer toute ma valeur ajoutée sur l'architecture et la résolution de problèmes complexes. Cette expérience m'a prouvé que la maîtrise technique d'un développeur moderne ne réside plus seulement dans sa connaissance d'un langage, mais dans sa capacité à orchestrer et à utiliser intelligemment les outils de son époque, IA incluse.

## **COMPÉTENCES**

**B1.6.2 Mettre en place son environnement d'apprentissage personnel : Configuration avancée de VSCode**

**B1.1.2 Recenser et identifier les ressources numériques : Sélection, installation et paramétrage des extensions de productivité.**

**B1.6.4 Mettre en œuvre des outils et stratégies de veille informationnelle.**